NETWORKING

Closing the Observability Loop in SAI

From Performance Telemetry to Platform Constraint Discovery

Jai Kumar *[Distinguished Engineer, Broadcom]*

Ravindranath Kanakarajan *[Distinguished Engineer, Marvell]*

OCP PLATINUM® Member

The problem: every switch differs — and applications can't see how

- AI workloads push ever-*tighter SLAs* onto *massive hyperscale switching* infrastructure
- Every vendor, family and silicon profile of NPUs expose different capabilities and hardware limits —
  - Even one NPU can switch ***profiles***, changing capabilities and scale
- No principled way exists to (1) discover those limits, or (2) know hardware programming cost
  - An ***opaque substrate*** that even advanced AI-driven controllers can't reason over or safely program

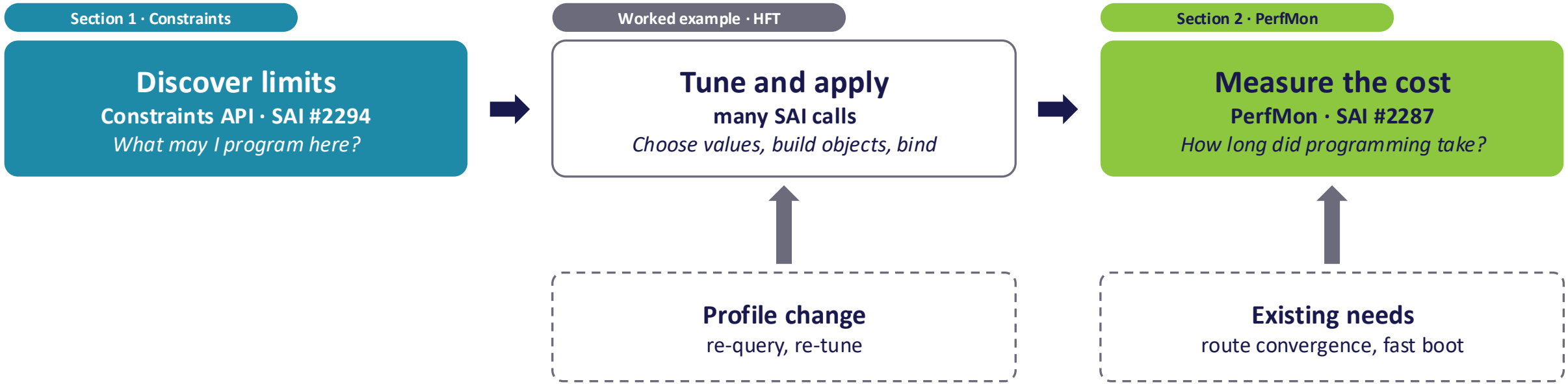
Scale-Up
GPU-to-GPU inside the rack — ultra-low latency

Scale-Out
Rack-to-rack, flattened multi-tier fabric

Scale-Across
Multi-row, multi-building reach over optics — e.g., OCI-class reach

Need standard ways to discover hardware limits and measure programming cost

Discover limits, tune, measure: one controller workflow



I. **Constraints API (#2294, in progress):** discover the limits the hardware enforces, before programming

II. **PerfMon (#2287, merged):** measure what programming costs — SAI API latency at the SDK boundary

SAI attribute model and Numeric constraints

Attribute based Object model

SAI Manages NPU as a set of logical objects

Attributes (an object's properties) are (id, value) pairs -- *extensibility*

Many of the attribute values are ***numeric*** *(uint32, int64 ...)*

A constraint limits what is legal

Varies based on NPU capability/profile

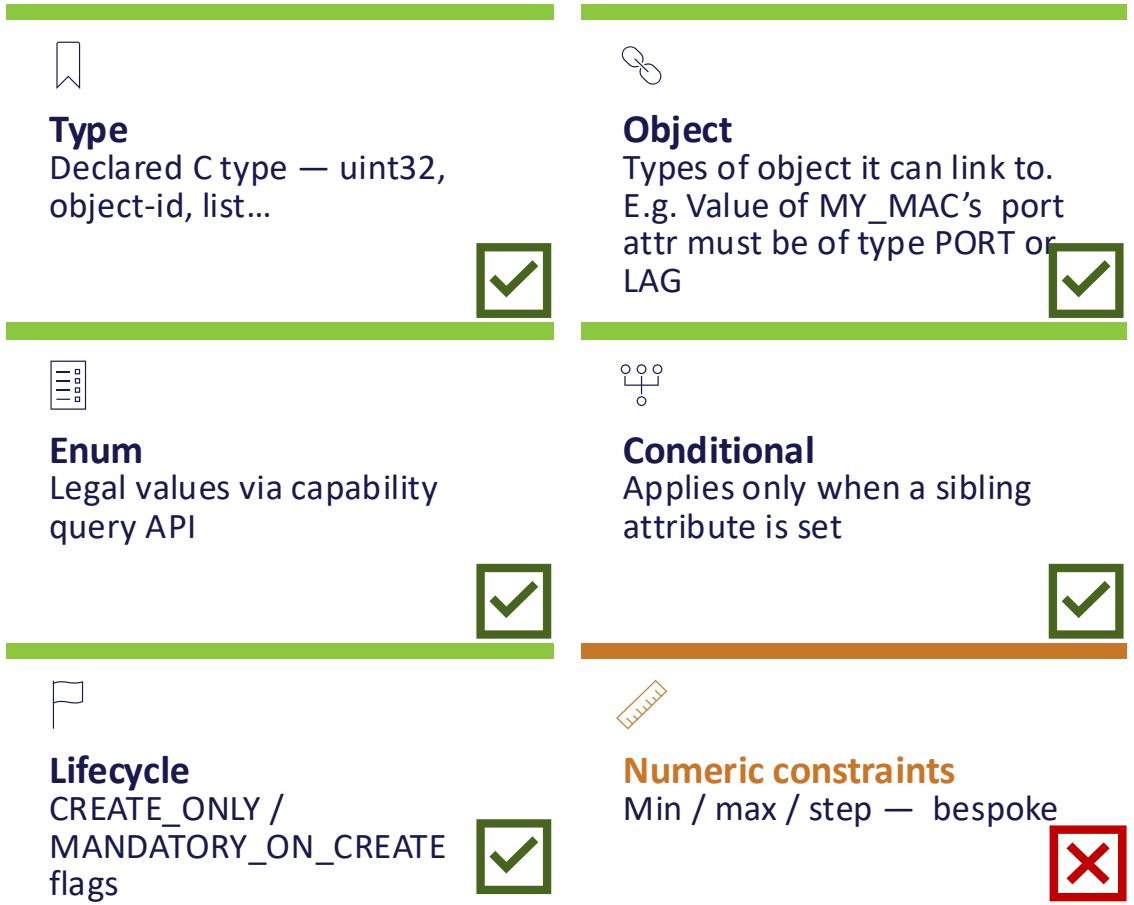
Some constraint types already modelled

Data-type, valid enumerations, lifecycle

Numeric constraints

Min/ max/ step-size

Modelled using bespoke attributes and is partial!



Bespoke model and its problems

Attribute	*Read-only* bespoke attribute	Data type
SAI_PORT_ATTR_SPEED	SAI_PORT_ATTR_SUPPORTED_SPEED	Discrete list of values
SAI_SWITCH_ATTR_PFC_TC_DLD_INTERVAL	SAI_SWITCH_ATTR_PFC_TC_DLD_INTERVAL_RANGE	Range (switch wide)
SAI_PORT_ATTR_PFC_TC_DLD_INTERVAL	SAI_PORT_ATTR_PFC_TC_DLD_INTERVAL_RANGE	Range (per-port)
Link Up Debounce timeout · Policer CIR · Metadata Range · etc.	**none**	—

Bloat
- *Per attribute, Per constraint*

Gaps
- Sporadic
- Partial constraints

Inconsistent
- Datatype
- Location
- Naming

Dependency
- Switch level
- Object level
- Before object creation
- Stage

Goal: Address these requirements with a unifying API

One generic API[1]

```
sai_status_t sai_query_attribute_value_constraints(
    _In_ sai_object_id_t switch_id,
    _In_ sai_object_type_t object_type,
    _In_ const sai_object_key_t *object_key,
    _In_ uint32_t attr_count,
    _In_ const sai_attribute_t *attr_list,
    _In_ sai_object_stage_t stage,
    _In_ sai_attr_id_t attr_id,
    _Inout_ sai_status_t *attr_status,
    _Inout_ sai_attribute_constraint_list_t *constraints);
```

■ context inputs ■ result channels

Multiple contexts

object_type — switch level dependencies

object_key — object level dependencies

attr_list — object level create-only dependencies

stage — INGRESS / EGRESS / BOTH, for limits set by a later binding

Returns List of Applicable Constraints — MIN | MAX | STEP | DEFAULT | ALLOWED_VALUES

Status and Discovering dependency params — attr_status | sai_status_t

Multiple dependencies + all applicable constraints — in one call

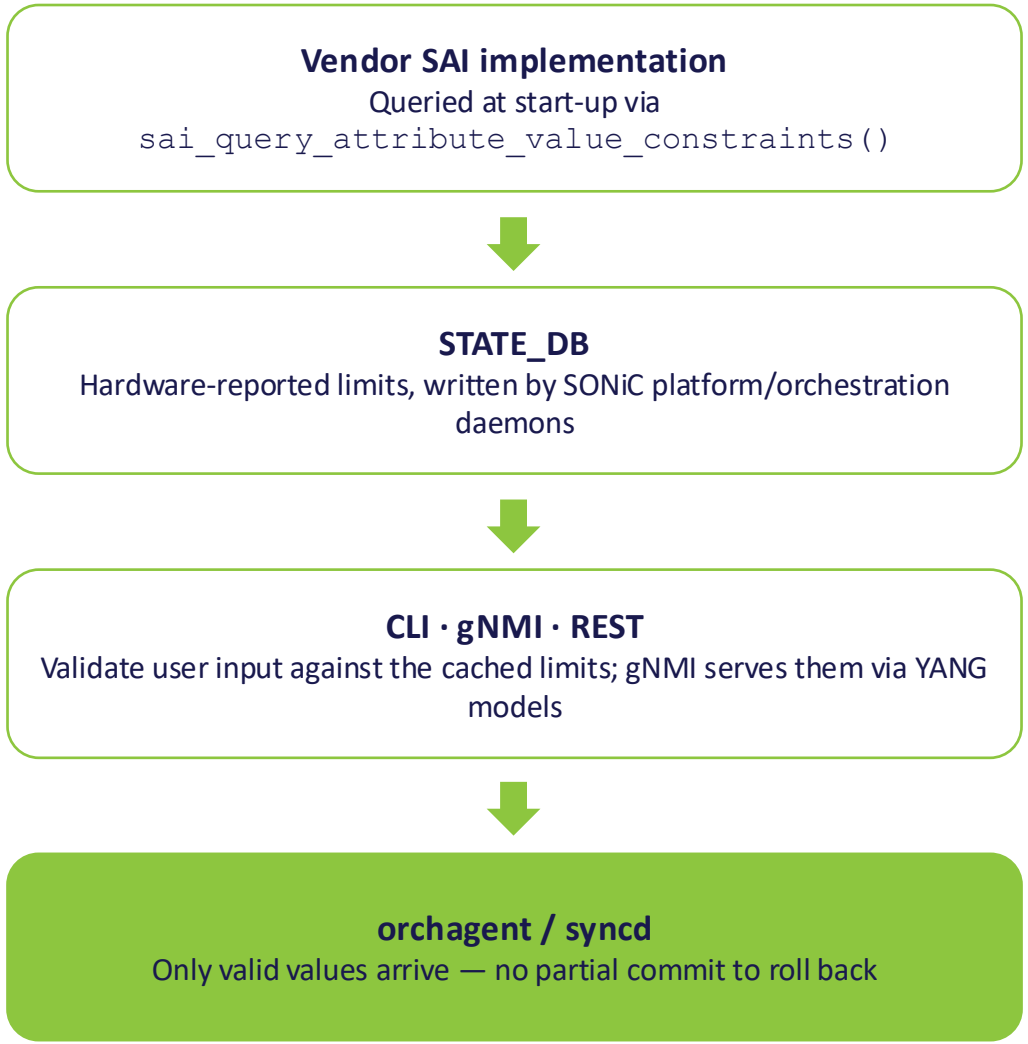
[1] Reference: SAI proposal #2294 (in review); subject to change

Examples

Use case	API Parameters (apart from attr_id)	Example attribute → returnback	Replaces today's…
Per Object-type	object_type	PORT_ATTR_LINK_UP_DEBOUNCE_TIMEOUT → MIN / MAX / STEP (µs)	A *_RANGE companion (e.g., SWITCH_ATTR_LINK_UP_ DEBOUNCE_ INTERVAL_RANGE)
Per instance	object_key	PORT_ATTR_SPEED → ALLOWED_VALUES: optical {100G, 200G, 400G}; copper {10G, 100G}	SAI_PORT_ATTR_SUPPORTED_SPEED
Per instance create-only	object_type + attr_list {POOL_ATTR_TYPE: INGRESS}	BUFFER_POOL_ATTR_SIZE* -> MAX	Cannot be expressed today
Sibling what-if	object_type + attr_list {METER_TYPE: BYTES}	POLICER_ATTR_CIR → MIN / MAX / STEP in **bps**	Cannot be expressed today
Pipeline stage	object_type + stage {EGRESS / INGRESS}	MIRROR_SESSION_ATTR_TRUNCATE_SIZE → MIN / MAX for that stage	Cannot be expressed today

A simple NOS use-case leveraging the ***uniformity*** of the API

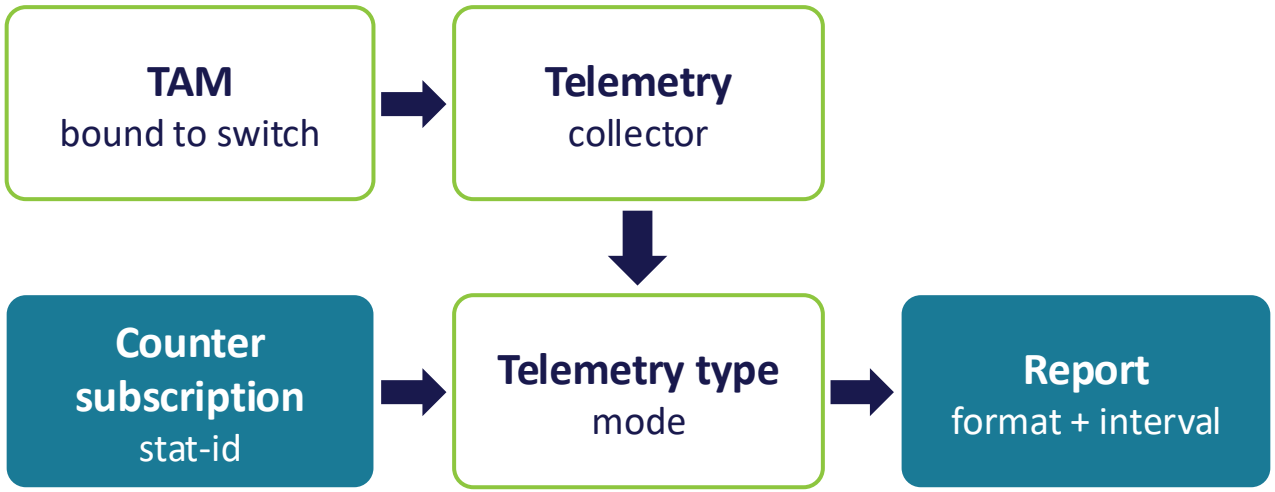
- **Discover once:** query the SAI implementation using just the object-type at start-up store the limits in *STATE_DB*
- **Uniformity**: Same API only the *object-type* and *attribute-id* varies; easy to implement and maintain
- **Validate early:** CLI, gNMI and REST check input against the hardware-reported limits
- **Avoid rollback:** invalid values never reach *orchagen*t or *syncd*, so nothing is partly committed
- Controllers read the same limits over gNMI before pushing configuration



Worked example: tuning High Frequency Telemetry (HFT)

- **HFT** streams selected stats from the vendor driver to a collector, at up to microsecond granularity
- **Today:** A dedicated API, `sai_query_stats_st_capability()` to get the fastest reporting interval per stat-id
- **Problem:** The minimum interval can also depend on the report's properties, not just the stat-id

Object hierarchy (arrow = refers to)



Interval depends on both: stat-id and report object's properties

Tuning sequence

1 Build exact plan but *don't bind/start*
Link the objects; start with a default interval

2 Query, then tune
Query the legal interval for format + stat-id; set it

3 Bind/start
Bind TAM to the switch; start stream; configuration takes effect

Build, tune and finally enable

Why control-plane API speed matters

Existing needs

Route convergence — after a failure or change, new routes must reach hardware promptly; bulk programming speed is part of recovery time

Fast / warm boot — restoring full hardware state at restart depends in part on SAI programming throughput

Scale — per-object costs multiply across thousands of ports, queues and flows

Emerging demands (projected)

Query-and-tune cycles — controllers that discover limits before configuring issue extra query, create and set calls per device

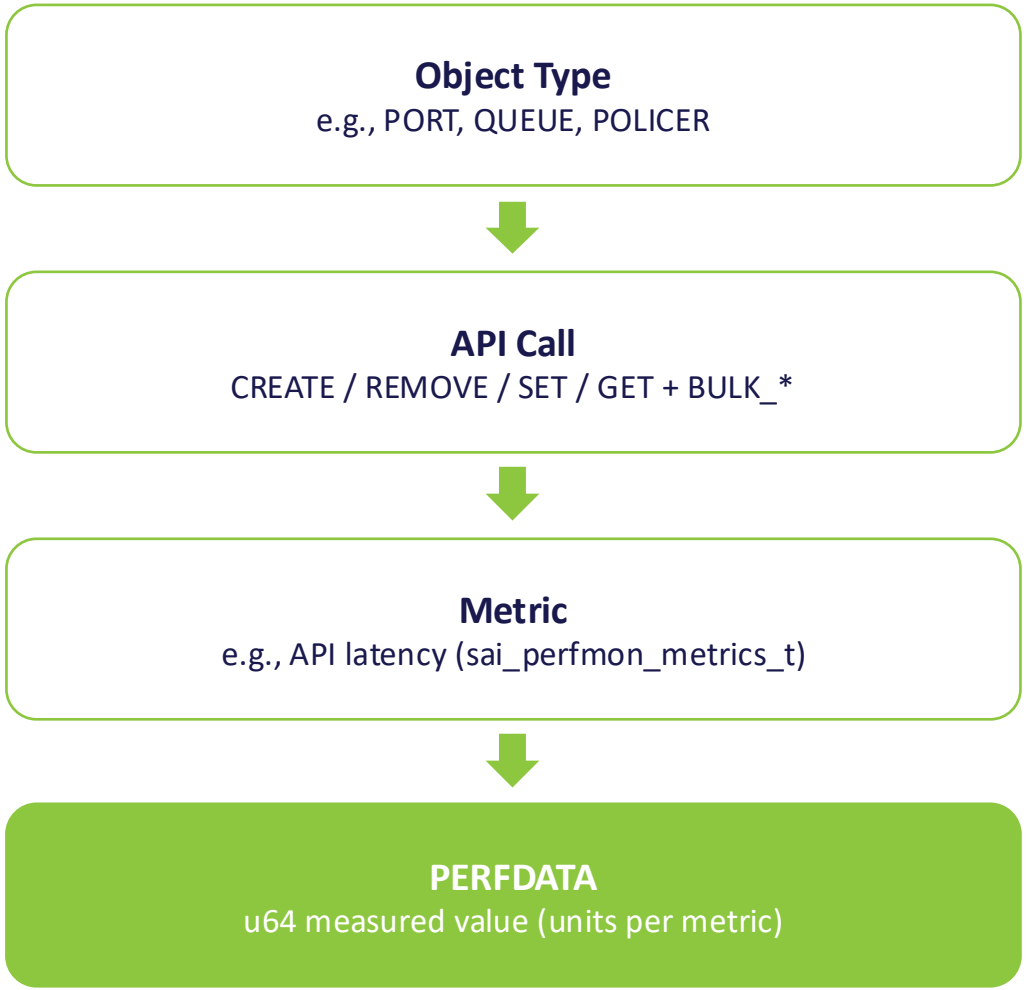
Profile changes — a new profile means re-discovering limits and re-programming, repeated across a fleet

Telemetry setup — streaming telemetry needs one subscription per object and stat: many objects to create before a stream starts

Common question: how long does each SAI call take, per object?

PerfMon: timing SAI calls inside the SDK

- New **SAI_OBJECT_TYPE_PERFMON** — merged into SAI master (#2287); builds on the 2023 OCP proposal
- Binds a target object type, an API call and a metric; reports SAI API latency at the SDK boundary
- **Timed in the SDK**: per-object latency inside BULK_* calls, free of caller-side variance and IPC/queueing



PerfMon workflow: timing the tune cycle

1 Configure
NOS creates perfmon objects: API call + metric

→

2 Program
Tune cycle: create, bulk-subscribe, set interval, bind

→

3 Measure
SDK timestamps inside each call, per object

→

4 Read
NOS reads SAI_PERFMON_ATTR_PERFDATA (u64)

→

5 Act
Publish via telemetry pipeline; controller paces or compares

Running example: the HFT tuning flow

Time it
Bulk subscription create (per object), report-interval set, TAM bind

Use it
Size the tune cycle, pace batches, compare SDK builds and vendors

Know the limit
SDK-observed time only; the query call itself is timed only if the PerfMon API enum covers it (to confirm)

Example consumption pattern for illustration.

Call to action

- Review **Query Platform Numerical Constraints on Supported Attribute Values** — opencomputeproject/SAI#2294
- Use **SAI API Performance Monitoring** — opencomputeproject/SAI#2287 (merged)
- **NOS developers:** consume both in open-source NOS — e.g., SONiC STATE_DB / gNMI
- **Controller & AI-agent builders:** query constraints before pushing configuration; use measured latency to pace it
- **Beyond Ethernet SAI:** any multi-vendor, object-based interface (e.g., UALink) faces the same blind spots — adopt early, avoid bespoke read-only attributes

Thank You — Questions?

